

Atividade 26 — Diagnóstico de Código, Refatoração e Desafio de Agendamento

Módulo 1 — Fundamentos de Backend, JavaScript Moderno e Node.js
Desenvolvimento Web — ADS — Unisenai

Acadêmico: Mauricio da Rosa Ventura

Projeto: node-profissional (continuação de hello-node, Módulo 0)

Data: 08/09/2026

1. Código original (fornecido pelo professor)

Código de `src/app.js` analisado nesta atividade, reproduzido na íntegra:

```
var x = [];  
  
var contador = 1;  
  
function fazer(a, b, c) {  
  if (a == undefined || a == "") {  
    console.log("erro");  
    return;  
  } else {  
    if (b == undefined || b == "") {  
      console.log("erro");  
      return;  
    } else {  
      if (c < 0) {  
        console.log("erro");  
        return;  
      }  
    }  
  }  
}  
  
var obj = {  
  id: contador,  
  nome: a,  
  duracao: b,  
  preco: c,  
  ativo: "sim"  
};  
  
contador = contador + 1;  
  
x.push(obj);  
  
console.log("cadastrado " + a);  
  
return obj;  
}  
  
function buscar(n) {  
  var achou = null;  
  
  for (var i = 0; i < x.length; i++) {  
    if (x[i].nome.toLowerCase() == n.toLowerCase()) {  
      achou = x[i];  
    }  
  }  
  
  return achou;  
}  
  
function listar() {  
  return x;  
}  
  
function desativar(id) {  
  for (var i = 0; i < x.length; i++) {  
    if (x[i].id == id) {  
      x[i].ativo = "nao";  
    }  
  }  
}  
  
function media() {  
  var soma = 0;  
  var quantidade = 0;  
  
  for (var i = 0; i < x.length; i++) {  
    if (x[i].ativo == "sim") {  
      soma = soma + x[i].preco;  
      quantidade++;  
    }  
  }  
  
  return soma / quantidade;  
}  
  
function lerConfig() {  
  try {  
    var porta = process.env.PORT || 3000;  
    console.log("porta: " + porta);  
  } catch (e) {  
  }  
}  
  
async function carregar() {  
  return Promise.resolve("dados carregados");  
}  
  
try {  
  fazer("Consulta", "45", 150);  
  fazer("consulta", 30, 100);  
}
```

```
fazer("", 50, 90);
fazer("Avaliação", -10, 120);

var servico = buscar("CONSULTA");

if (servico !== null) {
  console.log(servico.nome + " encontrado");
}

desativar("1");

console.log(listar());

console.log("Média: " + media());

lerConfig();

carregar().then(function (resultado) {
  console.log(resultado);
});

} catch (erro) {
  console.log("deu erro");
}
```

2. Problemas identificados (26.1)

Foram identificados 15 problemas — acima do mínimo de 10 pedido no enunciado — cobrindo nomenclatura, mutação, comparação, validação, organização, tratamento de erro, dependências/escopo, coerção de tipos, funções e arrays.

1. Nomes que não revelam intenção Nomenclatura

```
var x = [];

function fazer(a, b, c) { ... }
```

Por que é inadequado: Nomes de uma letra e o verbo genérico “fazer” não dizem o que a função faz nem o que cada parâmetro representa. Quem lê precisa executar o código mentalmente para descobrir que `a` é o nome do serviço, `b` a duração e `c` o preço.

Correção proposta: Nomes que revelam o domínio: `services` em vez de `x`, `createService(name, durationMinutes, price)` em vez de `fazer(a, b, c)`.

2. Uso de `var` em vez de `const / let` Variáveis e escopo

```
var x = [];
var contador = 1;
...
for (var i = 0; i < x.length; i++) { ... }
```

Por que é inadequado: `var` tem escopo de função (não de bloco) e sofre hoisting, o que facilita reatribuições acidentais e bugs sutis em loops. Nada nesse código muda de tipo ou de referência de um jeito que exija `var`.

Correção proposta: `const` como padrão; `let` apenas onde a referência realmente precisa mudar (contadores de `for`, acumuladores).

3. Comparação com `==` em vez de `===` Comparação / coerção de tipos

```
a == undefined
x[i].nome.toLowerCase() == n.toLowerCase()
x[i].ativo == "sim"
x[i].id == id
```

Por que é inadequado: `==` aplica coerção de tipo implícita (`0 == "0"` é `true`, `false == 0` é `true`), o que pode mascarar bugs de tipo — como o próprio problema #8 abaixo mostra.

Correção proposta: Usar `===` sempre, comparando valores já convertidos/normalizados na fronteira do sistema.

4. Validação incompleta, pouco expressiva e sem sinalização de erro Validação

```
if (a == undefined || a == "") {
  console.log("erro");
  return;
}
```

Por que é inadequado: Usa `==` para tratar “ausente”, não aplica `trim()` (aceita nome só com espaços), não valida o tipo/formato da duração (`c` só é checado como `< 0`, aceitando string, decimal ou `NaN`) e resolve a falha com `console.log + return` solto, sem lançar nada que identifique o problema.

Correção proposta: Uma função de validação dedicada (`service-validator.js`) que normaliza (`trim`), converte com `Number(...)` e usa `Number.isInteger / Number.isFinite`, lançando um `ValidationError` com mensagem específica por campo.

5. Mesmo campo guardado com tipos diferentes entre registros

Coerção de tipos

```
fazer("Consulta", "45", 150);    // duracao vira string "45"
fazer("consulta", 30, 100);     // duracao vira number 30
```

Por que é inadequado: Sem conversão explícita na fronteira, o mesmo campo `duracao` pode ser `string` em um registro e `number` em outro, quebrando qualquer operação numérica futura (soma, comparação, ordenação) que dependa desse campo.

Correção proposta: Converter com `Number(...)` dentro da validação e rejeitar o valor se não for um inteiro válido — garante um único tipo para todos os registros.

6. Flag booleana representada como string

Organização / boas práticas

```
ativo: "sim"
...
x[i].ativo = "nao";
```

Por que é inadequado: “sim”/“não” é um valor mágico que só funciona se todo o código concordar exatamente com a grafia (`"Sim"` ou `"SIM"` já quebrariam a comparação `==`), para representar um dado que é, por natureza, binário.

Correção proposta: Campo `active: true/false`, usando o tipo `boolean` nativo.

7. `listar()` expõe a referência interna do array

Mutação / arrays

```
function listar() {
  return x;
}
```

Por que é inadequado: Quem chama `listar()` recebe a mesma referência do array interno e pode fazer `listar()[0].preco = -999` ou `listar().push(...)` sem passar por nenhuma validação, quebrando o encapsulamento do módulo.

Correção proposta: Devolver sempre uma cópia — `services.map((service) => ({ ...service }))` — como em `service-repository.js`.

8. Busca não para no primeiro resultado e esconde um bug de tipo

Arrays / funções

```
for (var i = 0; i < x.length; i++) {
  if (x[i].nome.toLowerCase() == n.toLowerCase()) {
    achou = x[i];
  }
}
...
desativar("1"); // id "1" (string) contra id: 1 (number)
```

Por que é inadequado: O laço não usa `break` / `find` e por isso continua rodando após encontrar o item (desperdício, e se existissem nomes duplicados o último sempre “venceria” silenciosamente). Além disso, `desativar("1")` só funciona por acidente, porque `1 == "1"` é `true`; trocando por `===` (correção #3), esse bug de tipo escondido apareceria.

Correção proposta: Usar `Array.prototype.find` (para no primeiro resultado) e manter o *id* com um único tipo (string) do cadastro à busca, sem depender de coerção.

9. Nenhuma regra contra nomes duplicados

Regra de negócio

```
fazer("Consulta", "45", 150);
fazer("consulta", 30, 100); // aceito como serviço diferente
```

Por que é inadequado: O cadastro aceita “Consulta” e “consulta” como dois serviços diferentes, embora a própria `buscar()` os trate como o mesmo nome (comparação case-insensitive) — o comportamento de escrita e o de leitura são inconsistentes entre si, e o requisito do miniprojeto do módulo pede explicitamente essa unicidade.

Correção proposta: `createService` `consulta` `findByName` (case-insensitive) antes de salvar e lança `ConflictError` se o nome já existir.

10. Tratamento de erro só com `console.log("erro")`

Tratamento de erros

```
console.log("erro");
return;
```

Por que é inadequado: Não lança uma exceção, não diferencia o tipo de falha (nome vazio, duração inválida e preço negativo viram a mesma mensagem genérica “erro”) e devolve `undefined` silenciosamente — quem chamou `fazer(...)` não tem, por código, como saber que o cadastro falhou nem por quê.

Correção proposta: Lançar `ValidationError` com mensagem específica por regra, deixando quem chama decidir como reagir (`try/catch` em `app.js`).

11. `catch (e) {}` vazio Tratamento de erros

```
try {
  var porta = process.env.PORT || 3000;
  console.log("porta: " + porta);
} catch (e) {
}
```

Por que é inadequado: Um `catch` vazio esconde qualquer exceção que ocorra ali. Se o bloco algum dia lançasse (ex.: código adicionado depois), a falha desapareceria sem rastro, dificultando o diagnóstico — um anti-padrão citado explicitamente no material do módulo.

Correção proposta: Remover o `try/catch` (ler `process.env.PORT` não lança) ou, se necessário, tratar/registrar o erro de forma explícita.

12. Contador global mutável para gerar id Escopo / estado

```
var contador = 1;
...
contador = contador + 1;
```

Por que é inadequado: É estado mutável compartilhado por todo o módulo. Não é confiável se o processo reiniciar (o contador volta para 1 e pode colidir com ids já existentes) nem se houver mais de uma instância da aplicação rodando — cada uma teria seu próprio contador, gerando ids duplicados entre instâncias.

Correção proposta: Gerar um identificador que não dependa de estado compartilhado, como `crypto.randomUUID()` — a própria abordagem sugerida no material do módulo.

13. `media()` pode devolver `NaN` Funções / caso de borda

```
return soma / quantidade;
```

Por que é inadequado: Se não existir nenhum serviço ativo, `quantidade` é `0` e `soma / quantidade` vira `NaN`, que se propaga silenciosamente para qualquer lugar que use esse resultado (ex.: a string "Média: NaN").

Correção proposta: Checar `quantidade === 0` e devolver `0` antes de dividir, como em `getAverageActivePrice()`.

14. Uma única função concentra validação, regra, persistência e log Organização / dependências

```
function fazer(a, b, c) {
  // valida, decide, empurra em x e ainda faz console.log
}
```

Por que é inadequado: Misturar responsabilidades diferentes no mesmo bloco dificulta testar cada parte isoladamente (não dá para testar a regra de duração sem também acionar o `console.log` e o array global) e dificulta trocar qualquer uma delas (ex.: trocar `x` por PostgreSQL) sem tocar nas outras.

Correção proposta: Dividir em `service-validator.js` (validação), `service-repository.js` (persistência) e `service-service.js` (regra de negócio/orquestração).

15. `try/catch` externo genérico escondendo qualquer falha Tratamento de erros

```
try {
  fazer(...);
  ...
} catch (erro) {
  console.log("deu erro");
}
```

Por que é inadequado: Captura qualquer exceção — inclusive bugs de programação genuínos, como acessar uma propriedade de `undefined` — e reage sempre da mesma forma genérica, escondendo a causa real. O material do módulo é explícito: “não use `try/catch` para esconder falhas”.

Correção proposta: Tratar erros no ponto onde a camada sabe o que fazer com eles, diferenciando `ValidationError` / `ConflictError` (esperados) de um erro inesperado, que deve continuar se propagando em vez de ser engolido.

3. Refatoração (26.2)

A refatoração mantém o comportamento de domínio esperado — cadastrar, buscar, listar, desativar e calcular a média de serviços ativos — corrigindo os 15 problemas acima. Abaixo, os principais trechos antes/depois com a justificativa de cada decisão.

1. Variáveis e nomes

ANTES	DEPOIS
<pre>var x = []; var contador = 1; function fazer(a, b, c) { ... }</pre>	<pre>// service-repository.js const services = []; // service-service.js export function createService(input) { ... }</pre>

Justificativa: const/let em vez de var elimina hoisting e reatribuição accidental; nomes de domínio tornam o código legível sem precisar “rodar mentalmente” a função.

2. Comparação e flag ativo

ANTES	DEPOIS
<pre>if (x[i].ativo == "sim") { ... }</pre>	<pre>if (service.active) { ... }</pre>

Justificativa: === elimina coerção implícita; boolean nativo substitui o valor mágico "sim"/"nao", removendo uma fonte de bug de comparação.

3. Listagem sem expor estado interno

ANTES	DEPOIS
<pre>function listar() { return x; }</pre>	<pre>export function findAll() { return services.map((service) => ({ ...service })); }</pre>

Justificativa: Devolver cópias impede que quem chama mute o array interno do repositório por fora das regras de negócio.

4. Geração de id

ANTES	DEPOIS
<pre>var contador = 1; ... id: contador, contador = contador + 1;</pre>	<pre>import { randomUUID } from "node:crypto"; ... id: randomUUID(),</pre>

Justificativa: Remove o estado global mutável; um UUID não depende de nenhuma variável compartilhada nem se comporta diferente entre reinícios do processo ou múltiplas instâncias.

5. Tratamento de erro

ANTES	DEPOIS
<pre>if (a == undefined a == "") { console.log("erro"); return; } ... try { ... } catch (erro) { console.log("deu erro"); }</pre>	<pre>if (name === "") { throw new ValidationError("O nome do serviço é obrigatório."); } ... try { const service = createService(input); ... } catch (error) { if (error instanceof ApplicationError) { console.error(`Não cadastrado: \${error.message}`); return null; } throw error; // erro inesperado continua se propagando }</pre>

Justificativa: Erros passam a ser objetos identificáveis (ValidationError/ConflictError), tratados explicitamente onde a camada sabe reagir, sem esconder um bug de programação atrás de um catch genérico.

Estrutura de pastas final

```
src/
├── app.js                                # entrada: só conhece os casos de uso
└── modules/
    ├── services/
    │   ├── service-validator.js         # valida e normaliza a entrada (fronteira)
    │   ├── service-repository.js       # única parte que conhece a persistência
    │   └── service-service.js          # caso de uso: valida + regra + repositório
    ├── appointments/                   # desafio 26.3
    │   ├── appointment-validator.js
    │   ├── appointment-repository.js
    │   └── appointment-service.js      # regra de conflito de horário
    └── shared/
        └── errors/
            └── application-error.js     # ValidationError / ConflictError / NotFoundError
```

Código refatorado completo

src/modules/shared/errors/application-error.js

```
/**
 * Erro base da aplicação. Distinguir subclasses (em vez de usar Error puro
 * ou console.log) permite que cada camada decida como reagir: validação
 * vira resposta 400 no futuro, conflito vira 409, e um erro não reconhecido
 * continua se propagando em vez de ser silenciado.
 */
export class ApplicationError extends Error {
  constructor(message) {
    super(message);
    this.name = "ApplicationError";
  }
}

/** Entrada inválida (formato, tipo ou campo obrigatório ausente). */
export class ValidationError extends ApplicationError {
  constructor(message) {
    super(message);
    this.name = "ValidationError";
  }
}

/** Regra de negócio violada (ex.: nome duplicado, horário ocupado). */
export class ConflictError extends ApplicationError {
  constructor(message) {
    super(message);
    this.name = "ConflictError";
  }
}

/** Registro não encontrado para o identificador informado. */
export class NotFoundError extends ApplicationError {
  constructor(message) {
    super(message);
    this.name = "NotFoundError";
  }
}
```

src/modules/services/service-validator.js

```
import { ValidationError } from "../../shared/errors/application-error.js";

const MIN_SERVICE_DURATION_MINUTES = 1;

/**
 * Valida e normaliza a entrada de um serviço na fronteira do sistema.
 * Dados externos (aqui, os argumentos recebidos por createService) não são
 * confiáveis: convertemos tipos explicitamente (Number) e usamos === em vez
 * de == para não depender de coerção implícita.
 */
@param {{ name?: unknown, durationMinutes?: unknown, price?: unknown }} input
@returns {{ name: string, durationMinutes: number, price: number }}
export function validateServiceInput(input) {
  const name = typeof input?.name === "string" ? input.name.trim() : "";
  const durationMinutes = Number(input?.durationMinutes);
  const price = Number(input?.price);

  if (name === "") {
    throw new ValidationError("O nome do serviço é obrigatório.");
  }

  if (!Number.isInteger(durationMinutes) || durationMinutes < MIN_SERVICE_DURATION_MINUTES) {
    throw new ValidationError(
      "A duração do serviço deve ser um número inteiro maior que zero.",
    );
  }

  if (!Number.isFinite(price) || price < 0) {
    throw new ValidationError("O preço do serviço não pode ser negativo.");
  }
}
```

```
    return { name, durationMinutes, price };
  }
}
```

src/modules/services/service-repository.js

```
/**
 * Repositório em memória. É a única parte do módulo que conhece a forma
 * atual de "guardar" os dados – quando o PostgreSQL for introduzido, só
 * este arquivo muda; service-service.js continua igual.
 */
* Todas as funções devolvem cópias (spread), nunca a referência interna do
* array, para que quem chamar não consiga mutar o estado do repositório por
* fora (ex.: services[0].active = false direto na lista retornada).
*/
const services = [];

export function save(service) {
  services.push(service);
  return { ...service };
}

export function findAll() {
  return services.map((service) => ({ ...service }));
}

export function findById(id) {
  const found = services.find((service) => service.id === id);
  return found ? { ...found } : null;
}

export function findByName(name) {
  const normalizedName = name.toLocaleLowerCase("pt-BR");
  const found = services.find(
    (service) => service.name.toLocaleLowerCase("pt-BR") === normalizedName,
  );
  return found ? { ...found } : null;
}

export function update(id, changes) {
  const index = services.findIndex((service) => service.id === id);
  if (index === -1) {
    return null;
  }
  services[index] = { ...services[index], ...changes };
  return { ...services[index] };
}
```

src/modules/services/service-service.js

```
import { randomUUID } from "node:crypto";
import { validateServiceInput } from "../service-validator.js";
import { save, findAll, findByName, update } from "../service-repository.js";
import { ConflictError, NotFoundError } from "../../shared/errors/application-error.js";

/**
 * Caso de uso: cadastrar serviço. Orquestra validação + regra de negócio
 * (nome único, ignorando maiúsculas/minúsculas) + persistência, mas não
 * conhece detalhes de nenhuma das três – cada uma vive em seu próprio módulo.
 */
export function createService(input) {
  const data = validateServiceInput(input);

  if (findByName(data.name)) {
    throw new ConflictError(`Já existe um serviço cadastrado com o nome "${data.name}".`);
  }

  const service = {
    id: randomUUID(),
    ...data,
    active: true,
    createdAt: new Date().toISOString(),
  };

  return save(service);
}

export function listServices() {
  return findAll();
}

export function listActiveServices() {
  return findAll().filter((service) => service.active);
}

export function findServiceByName(name) {
  return findByName(name);
}

export function deactivateService(id) {
  const updated = update(id, { active: false });
  if (!updated) {
    throw new NotFoundError(`Nenhum serviço encontrado com o id "${id}".`);
  }
}
```



```

    return updated;
  }

  /**
   * Preço médio dos serviços ativos. Guard explícito contra divisão por zero
   * (0 / 0 = NaN em JS) quando não há nenhum serviço ativo.
   */
  export function getAverageActivePrice() {
    const active = listActiveServices();
    if (active.length === 0) {
      return 0;
    }
    const total = active.reduce((sum, service) => sum + service.price, 0);
    return total / active.length;
  }
}

```

src/app.js

```

import {
  createService,
  listServices,
  deactivateService,
  getAverageActivePrice,
  findServiceByName,
} from "../modules/services/service-service.js";
import { scheduleAppointment } from "../modules/appointments/appointment-service.js";
import { ApplicationError } from "../modules/shared/errors/application-error.js";

/**
 * app.js só conhece os casos de uso (createService, scheduleAppointment...).
 * Ele não sabe como a validação funciona nem como os dados são guardados –
 * essas responsabilidades ficam nos módulos de cada domínio.
 */

// -----
// 1. Cadastro de serviços (equivalente ao cenário original de app.js)
// -----

function tryCreateService(input) {
  try {
    const service = createService(input);
    console.log(`Serviço cadastrado: ${service.name} (id: ${service.id})`);
    return service;
  } catch (error) {
    if (error instanceof ApplicationError) {
      console.error(`Não cadastrado (${input?.name ?? "sem nome"}): ${error.message}`);
      return null;
    }
    throw error; // erro inesperado: não escondemos, deixamos propagar
  }
}

const consulta = tryCreateService({ name: "Consulta", durationMinutes: 45, price: 150 });
tryCreateService({ name: "consulta", durationMinutes: 30, price: 100 }); // nome duplicado -> ConflictError
tryCreateService({ name: "", durationMinutes: 50, price: 90 }); // nome vazio -> ValidationError
tryCreateService({ name: "Avaliação", durationMinutes: -10, price: 120 }); // duração inválida -> ValidationError

const encontrado = findServiceByName("CONSULTA");
if (encontrado) {
  console.log(`${encontrado.name} encontrado.`);
}

if (consulta) {
  deactivateService(consulta.id);
}

console.log(listServices());
console.log(`Média dos serviços ativos: ${getAverageActivePrice().toFixed(2)}`);

function readPort() {
  const port = Number(process.env.PORT ?? 3000);
  console.log(`Porta configurada: ${port}`);
  return port;
}
readPort();

async function loadData() {
  return "dados carregados";
}
console.log(await loadData());

// -----
// 2. Desafio 26.3 – agendamento em memória com validação de conflito
// -----

function tryScheduleAppointment(input) {
  try {
    const appointment = scheduleAppointment(input);
    console.log(
      `Agendamento criado: profissional ${appointment.professionalId}, ` +
      `serviço ${appointment.serviceId}, início ${appointment.startAt.toISOString()}`,
    );
    return appointment;
  } catch (error) {

```

```

    if (error instanceof ApplicationError) {
      console.error(`Agendamento recusado: ${error.message}`);
      return null;
    }
    throw error;
  }
}

tryScheduleAppointment({
  professionalId: "prof-1",
  serviceId: "service-1",
  startAt: "2026-09-10T14:00:00-03:00",
  durationMinutes: 45,
});

// mesmo profissional, horário sobreposto (14:20 cai dentro de 14:00-14:45) -> ConflictError
tryScheduleAppointment({
  professionalId: "prof-1",
  serviceId: "service-2",
  startAt: "2026-09-10T14:20:00-03:00",
  durationMinutes: 30,
});

// mesmo horário, profissional diferente -> permitido
tryScheduleAppointment({
  professionalId: "prof-2",
  serviceId: "service-1",
  startAt: "2026-09-10T14:00:00-03:00",
  durationMinutes: 45,
});

// duração inválida -> ValidationError
tryScheduleAppointment({
  professionalId: "prof-1",
  serviceId: "service-1",
  startAt: "2026-09-11T09:00:00-03:00",
  durationMinutes: 0,
});

```

Saída obtida ao rodar `npm start` (id e createdAt variam a cada execução; o restante é determinístico):

```

$ npm start
Serviço cadastrado: Consulta (id: ec151bbf-6dc2-4b6c-b278-d0586dfbf4e2)
Não cadastrado (consulta): Já existe um serviço cadastrado com o nome "consulta".
Não cadastrado (): O nome do serviço é obrigatório.
Não cadastrado (Avaliação): A duração do serviço deve ser um número inteiro maior que zero.
Consulta encontrado.
[
  {
    id: 'ec151bbf-6dc2-4b6c-b278-d0586dfbf4e2',
    name: 'Consulta',
    durationMinutes: 45,
    price: 150,
    active: false,
    createdAt: '2026-09-08T17:33:25.918Z'
  }
]
Média dos serviços ativos: 0.00
Porta configurada: 3000
dados carregados
Agendamento criado: profissional prof-1, serviço service-1, início 2026-09-10T17:00:00.000Z
Agendamento recusado: O profissional "prof-1" já possui um agendamento nesse horário.
Agendamento criado: profissional prof-2, serviço service-1, início 2026-09-10T17:00:00.000Z
Agendamento recusado: durationMinutes deve ser um número inteiro maior que zero.

```

4. Desafio 26.3 — agendamento em memória com validação de conflito

`scheduleAppointment` recebe `professionalId`, `serviceId`, `startAt` e `durationMinutes`, valida a entrada, calcula o horário de término e impede duração inválida e sobreposição de horário para o mesmo profissional. A regra foi decomposta em funções pequenas e puras (`calculateEndAt`, `periodsOverlap`) justamente para ficarem fáceis de entender e testar manualmente, como pedido no enunciado.

`src/modules/appointments/appointment-validator.js`

```
import { ValidationError } from "../../shared/errors/application-error.js";

/**
 * Valida a entrada de um agendamento. Segue o mesmo padrão de
 * service-validator.js: converte tipos explicitamente e falha cedo com uma
 * mensagem específica por campo, antes de qualquer regra de negócio.
 */
@param {{ professionalId?: unknown, serviceId?: unknown, startAt?: unknown, durationMinutes?: unknown }} input
*/
export function validateAppointmentInput(input) {
  const professionalId =
    typeof input?.professionalId === "string" ? input.professionalId.trim() : "";
  const serviceId = typeof input?.serviceId === "string" ? input.serviceId.trim() : "";
  const startAt = new Date(input?.startAt);
  const durationMinutes = Number(input?.durationMinutes);

  if (professionalId === "") {
    throw new ValidationError("professionalId é obrigatório.");
  }

  if (serviceId === "") {
    throw new ValidationError("serviceId é obrigatório.");
  }

  if (Number.isNaN(startAt.getTime())) {
    throw new ValidationError("startAt deve ser uma data/hora válida (ISO 8601).");
  }

  if (!Number.isInteger(durationMinutes) || durationMinutes <= 0) {
    throw new ValidationError("durationMinutes deve ser um número inteiro maior que zero.");
  }

  return { professionalId, serviceId, startAt, durationMinutes };
}
```

`src/modules/appointments/appointment-repository.js`

```
/**
 * Repositório de agendamentos em memória. Assim como em service-repository,
 * devolve sempre cópias e concentra a única referência mutável do módulo.
 */
const appointments = [];

export function save(appointment) {
  appointments.push(appointment);
  return { ...appointment };
}

export function findByProfessional(professionalId) {
  return appointments
    .filter((appointment) => appointment.professionalId === professionalId)
    .map((appointment) => ({ ...appointment }));
}

export function findAll() {
  return appointments.map((appointment) => ({ ...appointment }));
}
```

`src/modules/appointments/appointment-service.js`

```
import { randomUUID } from "node:crypto";
import { validateAppointmentInput } from "../appointment-validator.js";
import { save, findByProfessional } from "../appointment-repository.js";
import { ConflictError } from "../../shared/errors/application-error.js";

/**
 * Calcula o instante de término a partir do início e da duração.
 * Função pura, sem efeito colateral – fácil de testar isoladamente.
 */
export function calculateEndAt(startAt, durationMinutes) {
  return new Date(startAt.getTime() + durationMinutes * 60_000);
}

/**
 * Verifica se dois intervalos [start, end) se sobrepõem.
 * Função pura e pequena de propósito: é a regra central do desafio, então
 * precisa ser trivial de ler, testar e confiar.
 */
export function periodsOverlap(startA, endA, startB, endB) {
  return startA < endB && startB < endA;
}

/**
```

```

* Caso de uso: agendar um serviço com um profissional. A regra de conflito
* de horário mora aqui (no serviço de aplicação), não no repositório — o
* repositório só sabe persistir e consultar por profissional; quem decide
* se um horário é permitido é a regra de negócio, que também não deveria
* mudar quando trocarmos a persistência em memória pelo PostgreSQL.
*/
export function scheduleAppointment(input) {
  const data = validateAppointmentInput(input);
  const endAt = calculateEndAt(data.startAt, data.durationMinutes);

  const hasConflict = findByProfessional(data.professionalId).some((existing) => {
    const existingEnd = calculateEndAt(existing.startAt, existing.durationMinutes);
    return periodsOverlap(data.startAt, endAt, existing.startAt, existingEnd);
  });

  if (hasConflict) {
    throw new ConflictError(
      `O profissional "${data.professionalId}" já possui um agendamento nesse horário.`
    );
  }

  const appointment = {
    id: randomUUID(),
    ...data,
    endAt,
    createdAt: new Date().toISOString(),
  };

  return save(appointment);
}

```

Questões para discussão (propostas pelo material do módulo)

Onde deveria ficar a regra de conflito?

No serviço de aplicação (`appointment-service.js`), não no repositório nem no validador. É uma regra de negócio que depende de consultar outros registros — o validador só garante formato e tipo da entrada, sem conhecer os demais agendamentos.

O repositório deve decidir se um horário é permitido?

Não. O repositório só sabe persistir e consultar por profissional. Se ele decidisse sobre conflito, a regra ficaria acoplada aos detalhes de armazenamento e teria que ser reescrita quando a persistência mudasse.

O que acontecerá quando existirem duas requisições simultâneas?

Com o repositório em memória e chamadas síncronas, o event loop do Node processa uma chamada de `scheduleAppointment` por completo antes de iniciar a próxima, então não há problema aqui. Mas se essa função passasse a fazer `await` (ex.: uma consulta ao banco) entre a checagem de conflito e o `save`, duas requisições poderiam checar "livre" antes que qualquer uma delas salvasse, permitindo dois agendamentos conflitantes — uma condição de corrida. A checagem em memória não resolve isso sozinha.

Quais partes dessa solução mudarão quando o PostgreSQL for introduzido?

Só `appointment-repository.js` muda — de um array em memória para consultas SQL. `appointment-validator.js` e a regra de conflito em `appointment-service.js` continuam as mesmas. No banco, a checagem em código pode ainda ser reforçada por uma constraint (ex.: exclusão por intervalo/ `EXCLUDE` no PostgreSQL) como camada extra de segurança contra concorrência.

Exemplo de execução (trecho de `app.js`)

Um agendamento das 14h às 14h45 para `prof-1` é aceito; um segundo agendamento do mesmo profissional às 14h20 (dentro do intervalo anterior) é recusado com `ConflictError` ; o mesmo horário para `prof-2` é aceito, por ser outro profissional; e uma duração `0` é recusada na validação, antes de qualquer checagem de conflito — exatamente a saída reproduzida na seção 3.

5. Como executar

1. Dentro de `NODEJS/Mauricio-da-Rosa-Ventura/node-profissional/` : `npm install`
2. `npm start` (ou `npm run dev` para reiniciar automaticamente a cada alteração)
3. `npm run lint` / `npm run format` para checar e formatar o código

O README do projeto (`node-profissional/README.md`) detalha a estrutura e repete a saída esperada.